

FIRST COME FIRST SERVE

Program:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: \n");
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n];
```

```
    float total_tat = 0, total_wt = 0;
```

```
    // Input Arrival Time and Burst Time
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
    }
```

```
    // Calculate Completion Time (CT)
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (i == 0) {
```

```
            ct[i] = at[i] + bt[i];
```

```
        } else {
```

```
            // If process arrives after the previous one finishes
```

```
            int startTime = (ct[i - 1] > at[i]) ? ct[i - 1] : at[i];
```

```
            ct[i] = startTime + bt[i];
```

```
        }
```

```
    }
```

```
// Calculate Turnaround Time (TAT) and Waiting Time (WT)
for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
}

// Output results
printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
return 0;
}
```

Output:-

▼ Test Case - 1

Success 

Input :

```
3
0 2
1 2
2 3
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00
```

▼ Test Case - 2

Success 

Input :

```
3
0 5
5 3
8 2
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 0.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 0.00
```

NON-PREEMPTIVE SJF CPU SCHEDULING ALGORITHM

Program:-

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], tat[n], wt[n];
    int done[n];

    // Input
    for(int i = 0; i < n; i++)
    {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }

    int completed = 0;
    int completionTime = 0;
    float avgtat = 0, avgwt = 0;
    while(completed < n)//1
    {
        int idx = -1;
        int minBT = 9999;

        // Find shortest job among arrived processes
        for(int i = 0; i < n; i++)
        {
```

```

if(at[i] <= completionTime && done[i] == 0)
{
    if(bt[i] < minBT)
    {
        minBT = bt[i];
        idx = i;
    }
}
if(idx != -1)
{
    if(completionTime < at[idx]) {
        completionTime = at[idx];
    }
    completionTime += bt[idx];
    tat[idx] = completionTime - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    avgtat += tat[idx];
    avgwt += wt[idx];
    done[idx] = 1;
    completed++;
}
else
{
    // CPU idle
    completionTime++;
}

```

```

printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);

printf("Average Waiting Time: %.2f\n", avgwt / n);

return 0;
}

```

Output:-

Test Case - 1
Success

Input :

4
0 6
0 8
0 7
0 3

Expected Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

Test Case - 2
Success

Input :

3
0 3
5 2
5 1

Expected Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33
Average Waiting Time = 0.33

Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33
Average Waiting Time = 0.33

NON-PREEMPTIVE PRIORITY SCHEDULING ALGORITHM

Program:-

```
#include <stdio.h>

int main()
{
    int n, completed = 0, time = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int at[n], bt[n], priority[n], done[n];

    int ct[n], tat[n], wt[n];

    float avgtat = 0, avgwt = 0;

    // Input
    for (int i = 0; i < n; i++)
    {
        printf("Enter AT, BT and Priority for P%d: \n", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &priority[i]);
        done[i] = 0;
    }

    // Scheduling Logic
    while (completed < n)
    {
        int idx = -1;

        int highPriority = 9999;
```

```

for (int i = 0; i < n; i++)
{
    if (at[i] <= time && done[i] == 0)
    {
        if (priority[i] < highPriority)
        {
            highPriority = priority[i];
            idx = i;
        }
    }
}

if (idx != -1)
{
    time += bt[idx];
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    done[idx] = 1;
    completed++;
}
else
{
    time++; // CPU Idle
}
}

```



```
// Separate loop for average calculation
for (int i = 0; i < n; i++)
{
    avgtat += tat[i];
    avgwt += wt[i];
}
printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);
printf("Average Waiting Time: %.2f\n", avgwt / n);
return 0;
}
```

Output:-

▼ Test Case - 1

Success ✓

Input :

```
3
0 2
1 2
2 3
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00
```

▼ Test Case - 2

Success ✓

Input :

```
3
0 5
5 3
8 2
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 0.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 0.00
```

ROUND ROBIN CPU SCHEDULING ALGORITHM

Program:-

```
#include <stdio.h>

int main()
{
    int n, tq, completed = 0, completionTime = 0;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);
    int at[n], bt[n], rem[n], tat[n], wt[n];
    float avgtat = 0, avgwt = 0;
    // Input
    for (int i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: \n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i]; // Remaining time = Burst time
    }
    // Round Robin Logic
    while (completed < n)
    {
        int idle = 1;
        for (int i = 0; i < n; i++)
        {
            if (at[i] <= completionTime && rem[i] > 0)
            {
```

```

    idle = 0;
    if (rem[i] > tq)
    {
        completionTime += tq;
        rem[i] -= tq;
    }
    else
    {
        completionTime += rem[i];
        tat[i] = completionTime - at[i]; // Turnaround Time
        wt[i] = tat[i] - bt[i];        // Waiting Time
        rem[i] = 0;
        completed++;
    }
}

if (idle) {
    completionTime++; // If no process arrived yet
}

// Calculate averages
for (int i = 0; i < n; i++)
{
    avgtat += tat[i];
    avgwt += wt[i];
}

printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);
printf("Average Waiting Time: %.2f\n", avgwt / n);

```

```
    return 0;
}
```

Output:-

Test Case - 1

Success

Input :

3
2
0 5
1 3
2 1

Expected Output :

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

Output :

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

Test Case - 2

Success

Input :

4
3
0 4
1 5
2 2
4 1

Expected Output :

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:
Enter AT and BT for P4:

Average Turnaround Time: 8.00
Average Waiting Time: 5.00

Output :

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:
Enter AT and BT for P4:

Average Turnaround Time: 8.00
Average Waiting Time: 5.00

BANKER'S DEADLOCK AVOIDANCE ALGORITHM

Program:-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, r, i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &r);
```

```
    int alloc[n][r], max[n][r], need[n][r], avail[r];
```

```
    int finish[n], safe_sequence[n];
```

```
    int count = 0, safeFound;
```

```
    // Allocation Matrix
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < r; j++)
```

```
        {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
    }
```

```
// Max Matrix
printf("Enter Max Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < r; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

// Available Resources
printf("Enter Available Resources:\n");
for(i = 0; i < r; i++)
{
    scanf("%d", &avail[i]);
}

// Initialize finish array
for(i = 0; i < n; i++)
{
    finish[i] = 0;
}
```

```
// Calculate Need Matrix
for(i = 0; i < n; i++)
{
    for(j = 0; j < r; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
```

```
// Banker's Algorithm
while(count < n)
{
    safeFound = 0;
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int safe = 1;

            for(j = 0; j < r; j++)
            {
                if(need[i][j] > avail[j])
                {
                    safe = 0;
                    break;
                }
            }
        }
    }
}
```



```

    if(safe)
    {
        for(k = 0; k < r; k++)
        {
            avail[k] = avail[k] + alloc[i][k];
        }

        safe_sequence[count] = i;

        finish[i] = 1;

        safeFound = 1;

        count++;

    }
}

if(!safeFound)
{
    printf("System is in unsafe state! May occur the deadlock \n");

    return 0;
}

// Print Safe Sequence
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d ", safe_sequence[i]);
}

return 0;
}

```

Output:-

▼ Test Case - 1

Success ✓

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

▼ Test Case - 2

Success ✓

Input :

```
3
2
1 0
0 1
1 1
2 1
1 2
3 3
1 1
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is in a SAFE STATE.
Safe Sequence: P0 P1 P2
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is in a SAFE STATE.
Safe Sequence: P0 P1 P2
```

FIFO Page Replacement Algorithm

Program:-

```
#include <stdio.h>

int main() {
    int pages, frames;

    // Input
    printf("Enter number of pages in reference string: \n");
    scanf("%d", &pages);

    int ref[pages];
    printf("Enter the reference string: \n");
    for(int i = 0; i < pages; i++) {
        scanf("%d", &ref[i]);
    }

    printf("Enter number of frames: \n");
    scanf("%d", &frames);

    int frame[frames];

    // Initialize frames
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
    }

    int index = 0;
    int faults = 0, hits = 0;

    // FIFO Logic
    for(int i = 0; i < pages; i++) {
        int found = 0;

        // Check for hit
        for(int j = 0; j < frames; j++) {
```

```

        if(frame[j] == ref[i]) {
            found = 1;
            hits++;
            break;
        }
    }

    // If not found → Page Fault
    if(!found) {
        frame[index] = ref[i]; //frame size =4
        index = (index + 1) % frames;
        faults++;
    }
}

printf("\nTotal Page Faults: %d\n", faults);
printf("Total Page Hits: %d\n", hits);
return 0;
}

```

Output:-

▼ Test Case - 1

Success

Input :

7
1 3 0 3 5 6 3
3

Expected Output :

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 6
Total Page Hits: 1

Output :

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 6
Total Page Hits: 1

▼ Test Case - 2

Success

Input :

12
7 0 1 2 0 3 0 4 2 3 0 3
3

Expected Output :

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 10
Total Page Hits: 2

Output :

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 10
Total Page Hits: 2

LRU PAGE REPLACEMENT ALGORITHM

Program:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, frames;
```

```
    printf("Enter number of pages in reference string: \n");
```

```
    scanf("%d", &n);
```

```
    int pages[n];
```

```
    printf("Enter the reference string: \n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &pages[i]);
```

```
    }
```

```
    printf("Enter number of frames: \n");
```

```
    scanf("%d", &frames);
```

```
    int frame[frames], time[frames];
```

```
    // Initialize
```

```
    for(int i = 0; i < frames; i++) {
```

```
        frame[i] = -1;
```

```
        time[i] = 0;
```

```
    }
```

```
    int pageFaults = 0, pageHits = 0, counter = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        int found = 0;
```

```
        // Check for Hit
```

```

for(int j = 0; j < frames; j++) {
    if(frame[j] == pages[i]) {
        found = 1;
        pageHits++;
        counter++;
        time[j] = counter; // update recent use
        break;
    }
}

// Page Fault
if(!found) {
    int lruIndex = 0;
    // Find least recently used
    for(int j = 1; j < frames; j++) {
        if(time[j] < time[lruIndex]) {
            lruIndex = j;
        }
    }
    counter++;
    frame[lruIndex] = pages[i];
    time[lruIndex] = counter;
    pageFaults++;
}

}

printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits: %d\n", pageHits);
return 0;
}

```

Output:-

▼ Test Case - 1

Success ☑

Input :

Expected Output :

5
1 2 3 4 5
5

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0

Output :

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0

▼ Test Case - 2

Success ☑

Input :

Expected Output :

12
7 0 1 2 0 3 0 4 2 3 0 3
3

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 9
Total Page Hits: 3

Output :

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 9
Total Page Hits: 3

FIRST FIT CONTIGUOUS MEMORY ALLOCATION

Program:-

```
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int blockSize[m], remainingSize[m];
    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        remainingSize[i] = blockSize[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    for (int i = 0; i < n; i++) {
        int allocated = 0;
        for (int j = 0; j < m; j++) {
            if (remainingSize[j] >= processSize[i]) {
                int fragment = remainingSize[j] - processSize[i];
```

```
        printf("Process %d of size %d is allocated to Block %d of size %d with Fragment  
%d\n", i + 1, processSize[i], j + 1, blockSize[j], fragment);  
  
        remainingSize[j] -= processSize[i];  
  
        allocated = 1;  
  
        break;  
    }  
}  
  
if (!allocated) {  
    printf("Process %d of size %d is not allocated\n",  
        i + 1, processSize[i]);  
}  
}  
  
return 0;  
}
```

Output:-

Test Case - 1

Success 

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with
Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with
Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with
Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

Test Case - 2

Success 

Input :

```
4
50 100 200 300
3
70 90 150
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 70 is allocated to Block 2 of size 100 with
Fragment 30
Process 2 of size 90 is allocated to Block 3 of size 200 with
Fragment 110
Process 3 of size 150 is allocated to Block 4 of size 300 with
Fragment 150
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 70 is allocated to Block 2 of size 100 with Fragment 30
Process 2 of size 90 is allocated to Block 3 of size 200 with Fragment 110
Process 3 of size 150 is allocated to Block 4 of size 300 with Fragment 150
```

BEST FIT CONTIGUOUS MEMORY ALLOCATION

Program:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int m, n;
```

```
    printf("Enter number of memory blocks:\n");
```

```
    scanf("%d", &m);
```

```
    int blockSize[m], remaining[m];
```

```
    printf("Enter size of each block:\n");
```

```
    for (int i = 0; i < m; i++) {
```

```
        scanf("%d", &blockSize[i]);
```

```
        remaining[i] = blockSize[i];
```

```
    }
```

```
    printf("Enter number of processes:\n");
```

```
    scanf("%d", &n);
```

```
    int processSize[n];
```

```
    printf("Enter size of each process:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &processSize[i]);
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        int bestIndex = -1;
```

```
        for (int j = 0; j < m; j++) {
```

```

    if (remaining[j] >= processSize[i]) {

        if (bestIndex == -1 || remaining[j] < remaining[bestIndex]) {

            bestIndex = j;
        }
    }
}

if (bestIndex != -1) {

    int fragment = remaining[bestIndex] - processSize[i];

    printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
        i + 1, processSize[i], bestIndex + 1, blockSize[bestIndex], fragment);

    remaining[bestIndex] -= processSize[i];

} else {

    printf("Process %d of size %d is not allocated\n",
        i + 1, processSize[i]);
}

return 0;
}

```

Program:-

Test Case - 1

Success ☑

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with
Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with
Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with
Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with
Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```

Test Case - 2

Success ☑

Input :

```
3
100 200 300
4
90 150 250 400
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 90 is allocated to Block 1 of size 100 with
Fragment 10
Process 2 of size 150 is allocated to Block 2 of size 200 with
Fragment 50
Process 3 of size 250 is allocated to Block 3 of size 300 with
Fragment 50
Process 4 of size 400 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 90 is allocated to Block 1 of size 100 with Fragment 10
Process 2 of size 150 is allocated to Block 2 of size 200 with Fragment 50
Process 3 of size 250 is allocated to Block 3 of size 300 with Fragment 50
Process 4 of size 400 is not allocated
```